Лабораторна робота №10

Доповнення даних (Data Augmentation) для бінарної та мультикласової класифікації

Виконав: студент групи 443-4 **Бабин Богдан Олегович**

Мета роботи

Навчитися використовувати доповнення даних (Data Augmentation) разом з перенесенням навчання і тонким налаштуванням для бінарної класифікації (коні/люди, коти/собаки) та вирішувати задачі мультикласової класифікації (камінь-ножиці-папір, мова жестів) у Keras.

Посилання

- Завдання 1 (коні/люди, InceptionV3): https://colab.research.google.com/drive/1c3P8Qhc-hoiLQLgKprlSDb-OK2elt0oY
- Завдання 2 (тестові фото, 300×300, аугментація, fine tuning): https://colab.research.google.com/drive/1Xb-s2ge8OjW6l6EDsDKluqzZ4A6_ljuq
- Завдання 3a (коти/собаки, VGG16): https://colab.research.google.com/drive/1iLEb8mjau5H9wyPbkzMAyBwoacegXvRU
- Завдання 3b (коти/собаки, InceptionV3): https://colab.research.google.com/drive/1QQbwgkTOZpELvZFxMiqU60OQTv14xB5M
- Завдання 3c (коти/собаки, ResNet50): https://colab.research.google.com/drive/1uIM1O_8g8RZWvaGY43ZkBVrm1wGV-j1V
- Завдання 4 (Rock-Paper-Scissors): https://colab.research.google.com/drive/1F2DElddg7jjjnB67T3ay_IxsbV-7P39H
- Завдання 5 (Sign Language MNIST): https://colab.research.google.com/drive/12KZ2rqn7samMRvYSFWcnblbNVLCT3CB_
- Папка з усіма ноутбуками: https://drive.google.com/drive/folders/16CWF0mL5m9jHxNzXFwzqD6RY-zh58ZPR

Навчання — Google Colab, GPU T4. Збережені моделі: `lab10_2_horses_humans_aug.keras`, `lab10_3_<мережа>_best.keras`, `lab10_4_rps.keras`, `lab10_5_sign_mnist.keras` (завантажуються останньою коміркою ноутбуків).

Завдання 1. Перенесення навчання: коні / люди, InceptionV3

Код дописано лише між `### START CODE HERE` і `### END CODE HERE`. Генератори з `rescale=1./255` і `target_size=(150,150)`; InceptionV3 з локальними вагами, усі шари заморожені; вихід шару **mixed7** (7×7×768) → Flatten → Dense 1024 ReLU → Dropout 0,2 → Dense 1 sigmoid; модель створена через **Functional API** (`Model(inputs=pre_trained_model.input, outputs=x)`), RMSprop 1e-4, binary_crossentropy. Усі виводи збігаються з Expected Output: 21 802 784 параметри (0 навчаються), 47 512 481 / 38 537 217.

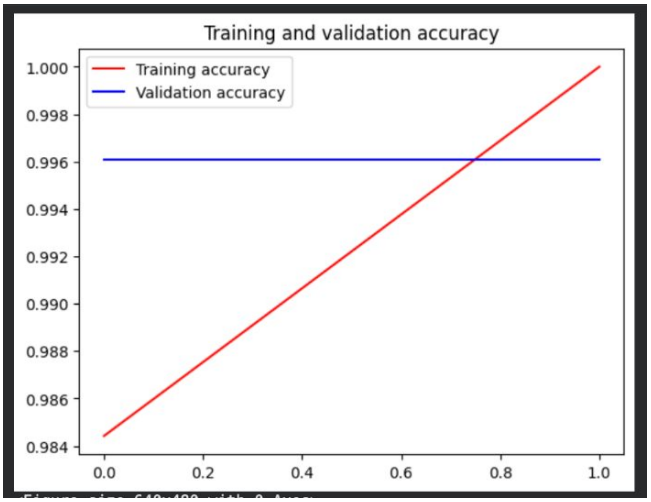
```
last_desired_layer = pre_trained_model.get_layer('mixed7')
```

```
last_output = last_desired_layer.output
x = layers.Flatten()(last_output)
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dropout(0.2)(x)
x = layers.Dense(1, activation='sigmoid')(x)
model = Model(inputs=pre_trained_model.input, outputs=x)
```

```
Epoch 1/100
33/33 - 16s - loss: 0.0339 - accuracy: 0.9844 - val_loss: 0.0130 - val_accuracy: 0.9961 - 16s/epoch - 476ms/step
Epoch 2/100

Досягнуто 99,9% точності, тому припиняємо навчання!
33/33 - 7s - loss: 6.1105e-04 - accuracy: 1.0000 - val_loss: 0.0107 - val_accuracy: 0.9961 - 7s/epoch - 219ms/step
```

Колбек зупинив навчання на 2-й епосі: train 100%, val 99,61%



Точність на навчальних і перевірочних даних

Технічна примітка: ноутбук написаний під Keras 2 (`layer.output_shape`, `w.shape.num_elements()`), а в Colab тепер Keras 3, тому в першій комірці вмикається пакет `tf_keras`. Посилання на ваги з методички (mledu-datasets) повертає 403, тому ті самі ваги InceptionV3 завантажено з офіційного сховища Keras.

Завдання 2. Тестові фото, роздільна здатність 300×300, аугментація, тонке налаштування

Чотири експерименти на основі ноутбука Завдання 1. Тестові фото — 7 коней і 7 людей зі фотобанку pixel-shot.com (немає в наборі) та 3 «обманні»: дівчина верхи, жінка з конем, жінка в костюмі вершниці. Особливість набору: навчальні фото — **згенеровані 3D-моделі**, тестові — справжні фотографії.

	експеримент	усього параметрів	навчаються	епох	час, с	с/епоху	train acc, %	val acc, %	найкраща val acc, %	val loss	нові фото	«обманні»
0	A. 150×150, без аугментації	47512481	38537217	2	24	11.8	100.0	99.22	99.61	0.0169	14/14	2/3
1	B. 300×300, без аугментації	236256161	227280897	2	27	13.4	100.0	97.27	98.05	0.1531	13/14	2/3
2	C. 150×150 + аугментація	47512481	38537217	7	95	13.5	99.9	98.05	100.00	0.0500	14/14	2/3
3	D. 150×150 + fine tuning (1 згортковий шар)	47512481	38684673	1	12	11.8	100.0	99.22	99.22	0.0182	14/14	2/3

Табл. Порівняння чотирьох експериментів



A. 150×150 без аугментації: 14 з 14 звичайних, 2 з 3 «обманних» (помилка — жінка поруч з конем)

Пункт 2. Роздільна здатність 300×300

```
Found 1027 images belonging to 2 classes.
Found 256 images belonging to 2 classes.
300×300: вихід mixed7 (None, 17, 17, 768)
Epoch 1/100
33/33 - 16s - loss: 0.5541 - accuracy: 0.9367 - val_loss: 0.1129 - val_accuracy: 0.9805 - 16s/epoch - 496ms/step
Epoch 2/100

Досягнуто 99,9% точності, тому скасовуємо навчання!
33/33 - 10s - loss: 1.8307e-04 - accuracy: 1.0000 - val_loss: 0.1531 - val_accuracy: 0.9727 - 10s/epoch - 306ms/step
```

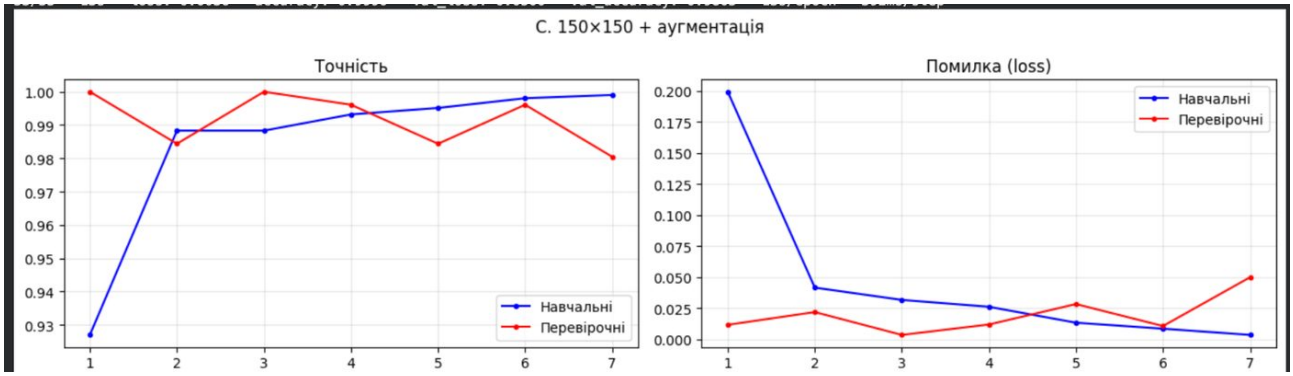
300×300: карта ознак mixed7 17×17×768 замість 7×7×768

Параметрів, що навчаються, стає **227 млн замість 38,5 млн** (у 6 разів більше — через Flatten 17·17·768 → Dense 1024). Навчання повільніше (13,4 проти 11,8 с/епоху), точність на перевірці нижча (97,3% проти 99,2%), втрати вищі (0,153 проти 0,017), на нових фото 13/14 (кінь-фото human_7 у студії → «кінь»). **Тестові фото розпізнаються гірше**: на ті самі 1027 зображень у 6 разів більше параметрів → сильніше перенавчання; деталізація 300×300 не допомагає, бо для розрізнення коня і людини достатньо загальної форми.

Пункт 3. Перенесення навчання + аугментація

```
Found 1027 images belonging to 2 classes.
Found 256 images belonging to 2 classes.
150×150: вихід mixed7 (None, 7, 7, 768)
Epoch 1/15
33/33 - 18s - loss: 0.1992 - accuracy: 0.9270 - val_loss: 0.0117 - val_accuracy: 1.0000 - 18s/epoch - 532ms/step
Epoch 2/15
33/33 - 13s - loss: 0.0416 - accuracy: 0.9883 - val_loss: 0.0220 - val_accuracy: 0.9844 - 13s/epoch - 387ms/step
Epoch 3/15
33/33 - 13s - loss: 0.0318 - accuracy: 0.9883 - val_loss: 0.0036 - val_accuracy: 1.0000 - 13s/epoch - 393ms/step
Epoch 4/15
33/33 - 12s - loss: 0.0262 - accuracy: 0.9932 - val_loss: 0.0120 - val_accuracy: 0.9961 - 12s/epoch - 373ms/step
Epoch 5/15
33/33 - 13s - loss: 0.0134 - accuracy: 0.9951 - val_loss: 0.0284 - val_accuracy: 0.9844 - 13s/epoch - 394ms/step
Epoch 6/15
33/33 - 13s - loss: 0.0086 - accuracy: 0.9981 - val_loss: 0.0108 - val_accuracy: 0.9961 - 13s/epoch - 388ms/step
Epoch 7/15

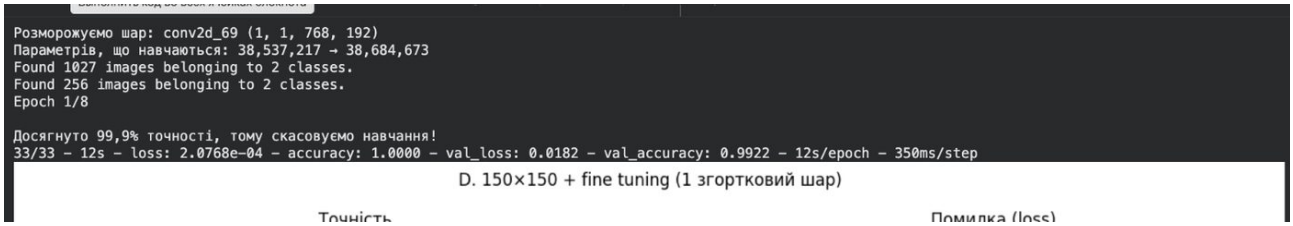
Досягнуто 99,9% точності, тому скасовуємо навчання!
33/33 - 13s - loss: 0.0036 - accuracy: 0.9990 - val_loss: 0.0500 - val_accuracy: 0.9805 - 13s/epoch - 392ms/step
```



Аугментація: точність на перевірці коливається 98–100%

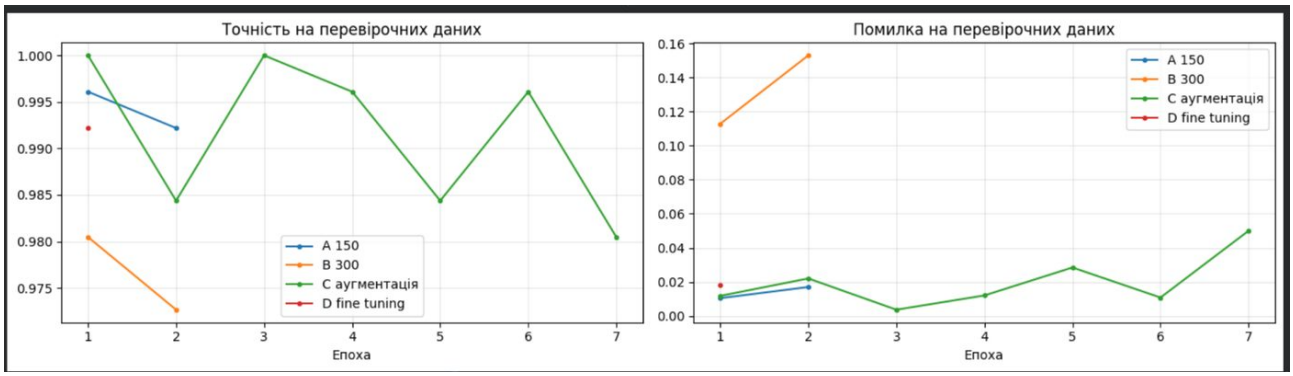
Навчання триває довше (7 епох замість 2): кожне фото щоепохи інше, тому 99,9% навчальної точності досягається пізніше. Точність на перевірці коливається 98–100%, на нових фото — так само 14/14 і 2/3. **Результат пов'язаний з особливостями набору**: навчальні й перевірочні фото — одні й ті самі 3D-моделі на однотонному фоні, тому перевірка дуже легка (99%+ вже без аугментації), і аугментації нема чого покращувати. Перевірочний набір не відображає справжніх фото, а InceptionV3 і так знає справжніх коней з ImageNet.

Пункт 4. Перенесення навчання + тонке налаштування



Розморожено ще один згортковий шар conv2d_69 (147 456 параметрів)

Розморожено останній згортковий шар перед mixed7 (`conv2d_69`, 1×1×768×192), RMSprop 1e-5. Колбек спрацював уже на 1-й епосі (точність і так 100%), результат такий самий: val 99,22%, нові фото 14/14, 2/3. Модель уже ідеально розділяє навчальні фото, тому донавчання одного шару нічого не змінює.



Точність і помилка на перевірочних даних для чотирьох експериментів

файл	справжній	A	B	C	D
horse_1.jpg	кінь	кінь	кінь	кінь	кінь
horse_2.jpg	кінь	кінь	кінь	кінь	кінь
horse_3.jpg	кінь	кінь	кінь	кінь	кінь
horse_4.jpg	кінь	кінь	кінь	кінь	кінь
horse_5.jpg	кінь	кінь	кінь	кінь	кінь
horse_6.jpg	кінь	кінь	кінь	кінь	кінь
horse_7.jpg	кінь	кінь	кінь	кінь	кінь
horse_h1.jpg	кінь	кінь	кінь	кінь	кінь
horse_h2.jpg	кінь	людина	кінь	людина	людина
human_1.jpg	людина	людина	людина	людина	людина
human_2.jpg	людина	людина	людина	людина	людина
human_3.jpg	людина	людина	людина	людина	людина
human_4.jpg	людина	людина	людина	людина	людина
human_5.jpg	людина	людина	людина	людина	людина
human_6.jpg	людина	людина	людина	людина	людина
human_7.jpg	людина	людина	кінь	людина	людина

Терминал

Відповіді чотирьох моделей на кожне фото

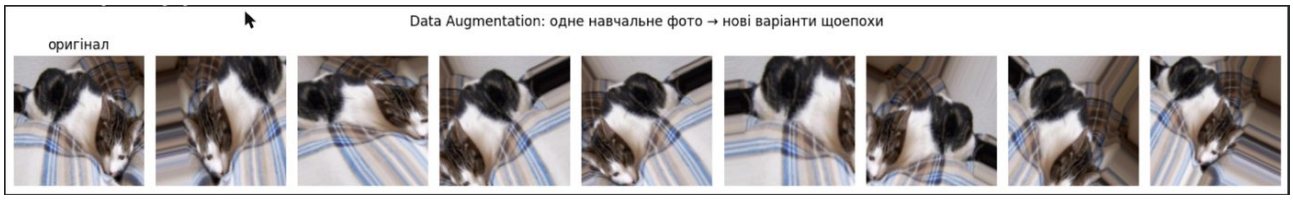
«Обманне» фото **horse_h2** (жінка з конем) три моделі з чотирьох вважають людиною — на ньому справді є обидва класи, а людина ближче до камери. Розпізнає його лише модель 300×300, але вона ж помиляється на human_7 і human_h1.

Завдання 3. Коти / собаки (2000 / 500 / 500): аугментація + тонке налаштування

Пункти 5–6. Скрипт з лаби 7 змінено: з кожного класу відкидаються пошкоджені фото, після перемішування беруться рівно 3000 різних фото — 2000 у train, 500 у val, 500 у test (перевірка `assert len(set(chosen)) == len(chosen)`).

```
train {'cats': 2000, 'dogs': 2000}    val {'cats': 500, 'dogs': 500}    test {'cats': 500, 'dogs': 500}
```

Пункти 7–9. Навчальний генератор — з параметрами аугментації з методички (rotation 40, зсуви 0,2, zoom 0,2, horizontal flip, fill nearest); генератор для перевірки і тесту — без аугментації. Замість `rescale` — `preprocess_input` відповідної мережі (як у лабах 8–9).



Одне навчальне фото і 8 випадкових варіантів, які генерує аугментація

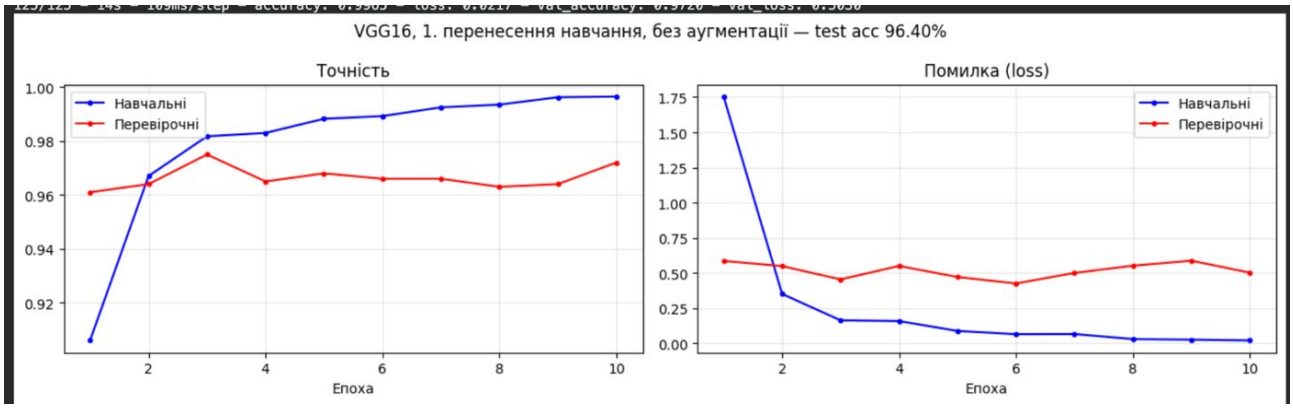
Пункти 10–12. VGG16

Класифікатор — Flatten → Dense 256 → Dropout 0,5 → sigmoid, Adam 1e-4 (найкраща конфігурація з лаби 8). Усі експерименти стартують з однакових ваг.

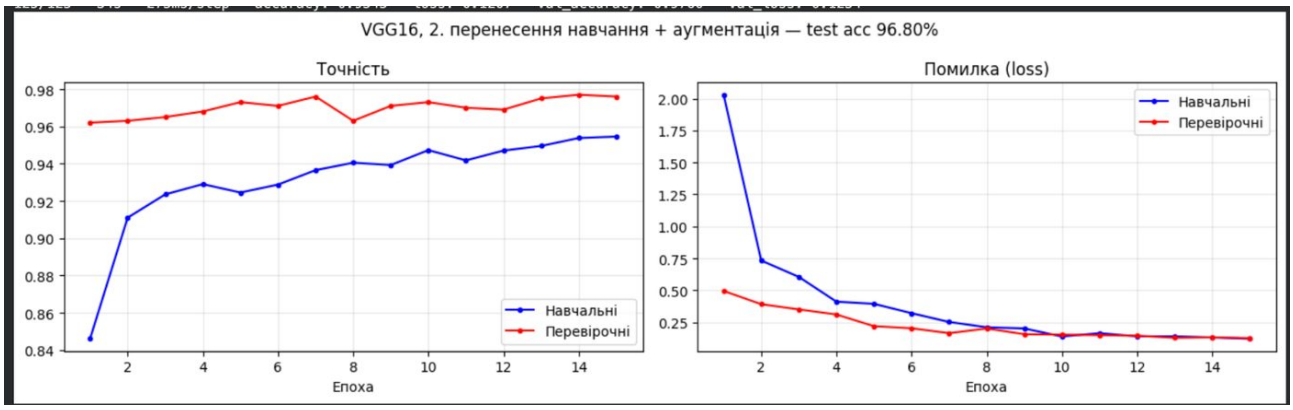
Експеримент	Навч. параметрів	Епох	Train acc	Val acc	Розрив	Val loss	Test acc, %	Test loss	Час, с
1. Перенесення навчання, без аугментації	2097665	10	99,65	97,2	2,45	0,503	96,4	0,484	151

Експеримент	Навч. параметрів	Епох	Train acc	Val acc	Розрив	Val loss	Test acc, %	Test loss	Час, с
2. Перенесення навчання + аугментація	2097665	15	95,45	97,6	-2,15	0,125	96,8	0,134	537
3. Аугментація + fine tuning 1 блоку	9177089	4	96,38	98,1	-1,73	0,138	97,4	0,157	150
4. Аугментація + fine tuning 2 блоків	15076865	10	98,35	98,1	0,25	0,135	**97,6**	0,104	380
5. Перенесення навчання + слабша аугментація	2097665	15	98,33	97,1	1,23	0,263	97,2	0,275	528

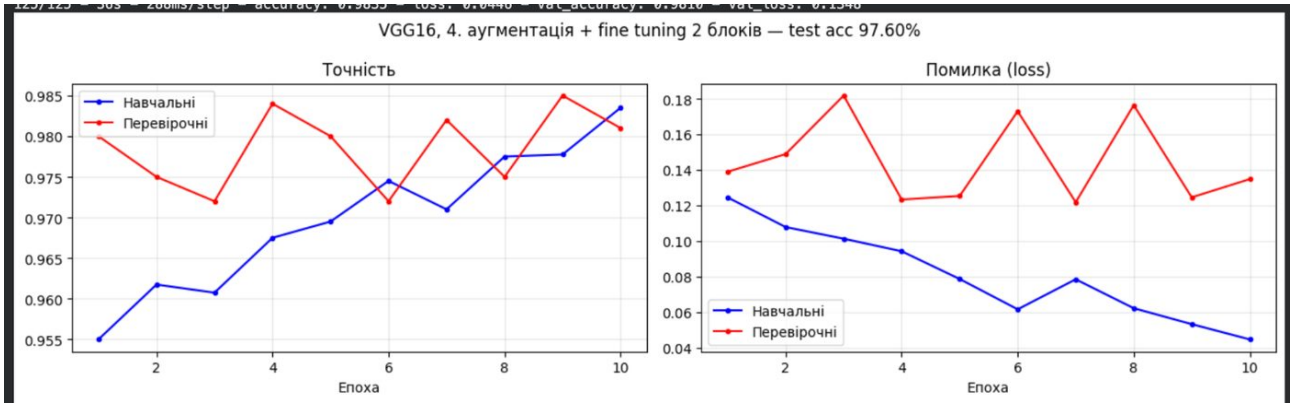
Табл. VGG16: результати експериментів (тест — 1000 фото)



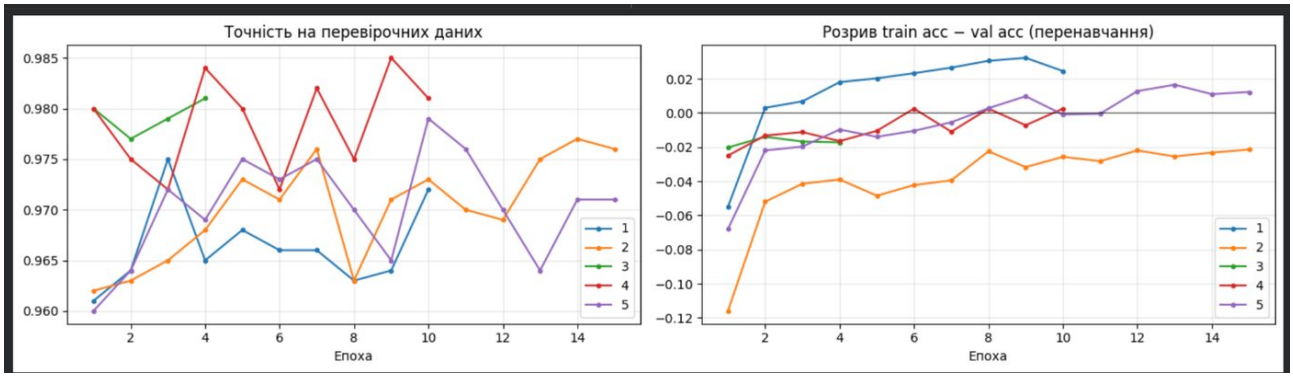
1. Без аугментації: train 99,7%, а val застрягла на 96–97%, втрати на перевірці 0,5 — перенавчання



2. З аугментацією: train нижча за val, втрати на перевірці падають до 0,13



4. Аугментація + fine tuning 2 блоків — найкраща модель VGG16 (97,6%)



Точність на перевірці і розрив train − val для п’яти експериментів

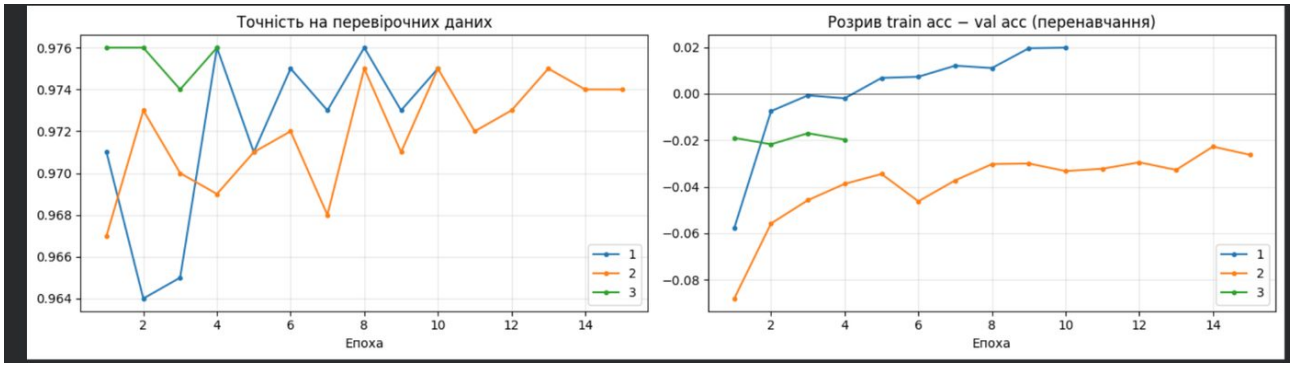
Що показали експерименти. Без аугментації модель за 10 епох запам’ятовує навчальні фото (99,7%), а на перевірці втрати ростуть до 0,5 — класичне перенавчання. З аугментацією навчальна точність **нижча** за перевірочну (95,5% проти 97,6%): модель щоепохи бачить нові варіанти фото і не встигає їх завчити, втрати на перевірці падають у 4 рази (0,50 → 0,13), тест +0,4%. Ціна — час: епоха 35 с замість 14 с (картинки трансформуються на процесорі). Fine tuning поверх аугментації дає ще +0,6…0,8%: 1 блок — 97,4%, 2 блоки — **97,6%**, найкраще для VGG16. Слабша аугментація (поворот 15°, зсуви 0,1) знову повертає розрив train − val (+1,2%) і вищі втрати 0,26 — тобто саме сильна аугментація з методички найкраще бореться з перенавчанням.

Пункт 12. InceptionV3 і ResNet50

Класифікатори — найкращі з лаби 9: InceptionV3 — GAP → Dense 512 → Dropout 0,5; ResNet50 — GAP → Dense 32 → Dropout 0,6. Ті самі експерименти 1–3.

Експеримент	Навч. параметрів	Епох	Train acc	Val acc	Розрив	Val loss	Test acc, %	Test loss	Час, с
1. Перенесення навчання, без аугментації	1049601	10	99,48	97,5	1,97	0,076	96,3	0,107	153
2. Перенесення навчання + аугментація	1049601	15	94,77	97,4	-2,63	0,068	**97,5**	0,076	536
3. Аугментація + fine tuning 1 блоку (mixed10)	7119873	4	95,63	97,6	-1,97	0,067	**97,5**	0,073	164

Табл. InceptionV3

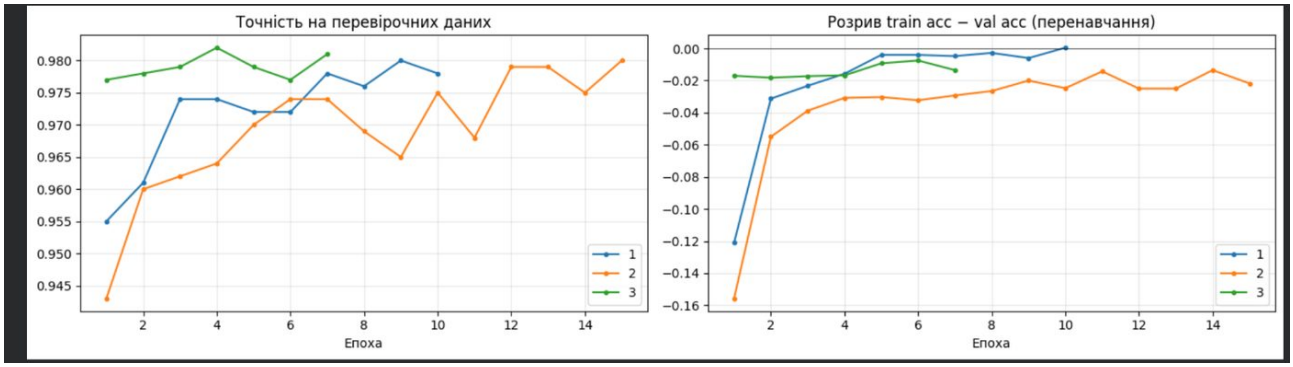


InceptionV3: точність на перевірці і розрив train − val

Експеримент	Навч. параметрів	Епох	Train acc	Val acc	Розрив	Val loss	Test acc, %	Test loss	Час, с
1. Перенесення навчання, без аугментації	65601	10	97,85	97,8	0,05	0,066	**97,4**	0,069	151

Експеримент	Навч. параметрів	Епох	Train acc	Val acc	Розрив	Val loss	Test acc, %	Test loss	Час, с
2. Перенесення навчання + аугментація	65601	15	95,82	98,0	-2,18	0,073	97,3	0,073	521
3. Аугментація + fine tuning 1 блоку (conv5_block3)	4525121	7	96,75	98,1	-1,35	0,072	**97,4**	0,067	268

Табл. ResNet50

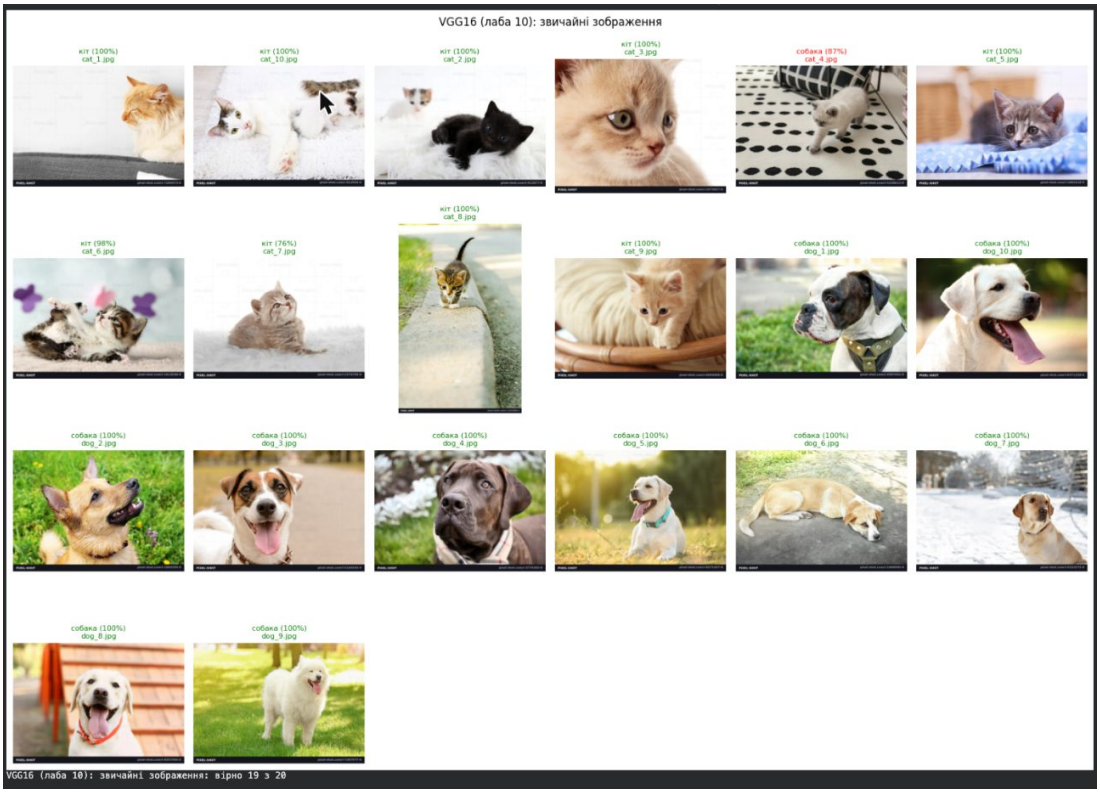


ResNet50: розрив майже нульовий уже без аугментації

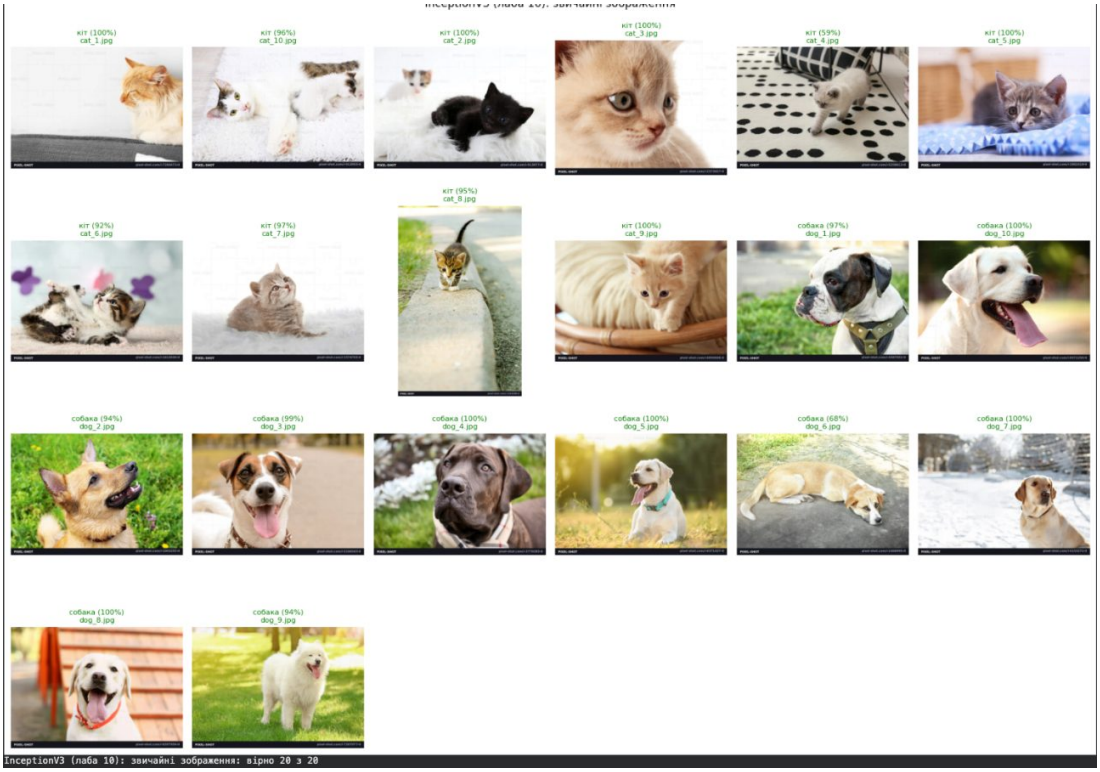
InceptionV3: без аугментації — та сама картина, що у VGG16 (train 99,5%, розрив +2%, тест 96,3%); аугментація прибирає розрив і дає **+1,2% на тесті (97,5%)**, втрати на тесті падають з 0,107 до 0,076. Fine tuning mixed10 зупинився на 4-й епосі з тим самим тестом 97,5%. **ResNet50** поводиться інакше: класифікатор крихітний (Dense 32 + Dropout 0,6, лише 65 601 параметр), тому вона не перенавчається і без аугментації (розрив 0,05%) — аугментація тут нічого не додала (97,3…97,4%), а fine tuning conv5_block3 підняв перевірку до 98,1%, але тест той самий 97,4%. Висновок: аугментація найбільше допомагає моделям з великим класифікатором (VGG16 — 2,1 млн, InceptionV3 — 1 млн параметрів), які схильні завчувати фото.

Пункт 13. Розпізнавання тестових фото і порівняння з лабами 7–9

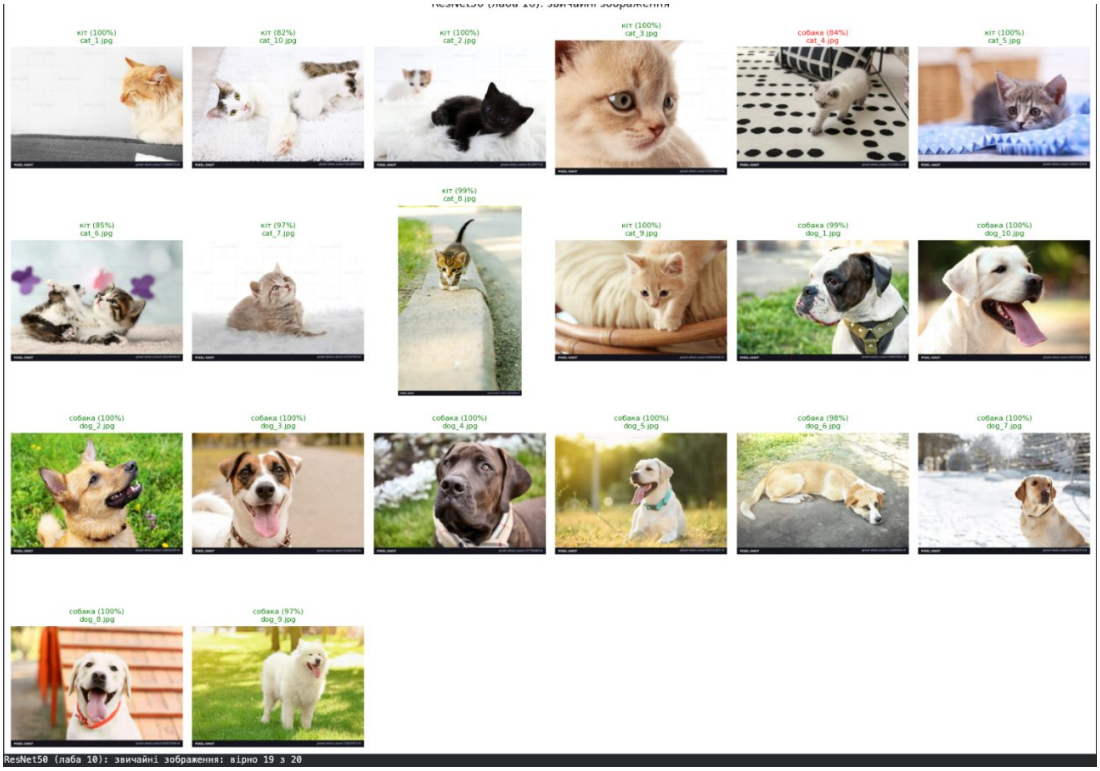
Власні фото з лаб 7–9 цього разу завантажити в Colab не вдалося (розширення браузера перестало пропускати локальні файли), тому найкращі моделі перевірено на **20 нових фото** котів і собак з фотобанку pixel-shot.com (10 + 10), яких немає в наборі Dogs vs Cats. Комірка автоматично використає власний архів `lab8_test_images.zip`, якщо його завантажити через панель «Файли».



VGG16 (аугментація + fine tuning 2 блоків): 19 з 20 (помилка — кошеня cat_4 на рябому килимі)



InceptionV3 (аугментація): 20 з 20 — навіть кошеня cat_4 розпізнане (59%)



ResNet50 (без аугментації): 19 з 20 (помилка — те саме кошеня cat_4)

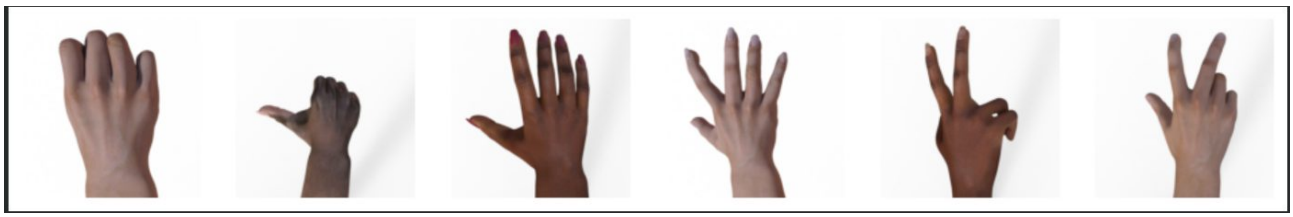
Робота	Модель / технологія	Навчальних фото	Test acc, %
Лаба 7	власна CNN з нуля	17 500	84,75
Лаба 8	VGG16, перенесення навчання (Dropout 0,5)	4 200	97,56
Лаба 9	InceptionV3, перенесення + підбір гіперпараметрів	4 200	98,00
Лаба 10	VGG16, без аугментації	4 000	96,4
Лаба 10	VGG16, аугментація + fine tuning 2 блоків	4 000	97,6
Лаба 10	InceptionV3, найкраща	4 000	97,5
Лаба 10	ResNet50, найкраща	4 000	97,4

Табл. Порівняння з лабораторними 7–9 (тестові набори різні за розміром, тому порівняння орієнтовне)

На меншому наборі (4000 навчальних фото проти 4200 у лабах 8–9 і 17 500 у лабі 7) аугментація з fine tuning дала VGG16 **97,6%** — на рівні лаби 8 (97,56%) при меншій кількості даних, а InceptionV3 — 97,5% (у лабі 9 — 98,0% на більшому наборі). Усі три мережі на 12–13% краще за власну CNN з лаби 7 (84,75%), яка вчилась на вчетверо більшому наборі, — перенесення навчання важливіше за кількість даних, а аугментація дозволяє обійтися меншим набором без перенавчання. Різниця між мережами в межах 0,3% (кілька фото з 1000), тобто на цьому тесті вони практично рівні. На 20 нових фото з інтернету: VGG16 — 19/20, InceptionV3 — 20/20, ResNet50 — 19/20 (усі помилки — кошеня cat_4 на рябому килимі).

Завдання 4. Мультикласова класифікація: камінь-ножиці-папір

Базовий ноутбук без змін у моделі: 4 згорткові шари (64-64-128-128) → Dropout 0,5 → Dense 512 → **Dense 3 softmax**, **categorical_crossentropy**, RMSprop, аугментація навчальних фото, 25 епох. Відмінність від бінарної задачі — 3 виходи з softmax (ймовірності трьох класів, сума = 1) і one-hot мітки (`class_mode='categorical'`). Порядок класів: paper, rock, scissors. У базовій комірці розпізнавання не було ділення на 255 — додано, інакше вхід не збігається з навчальним.



Навчальні фото: 3D-моделі рук на білому фоні (840 на клас)

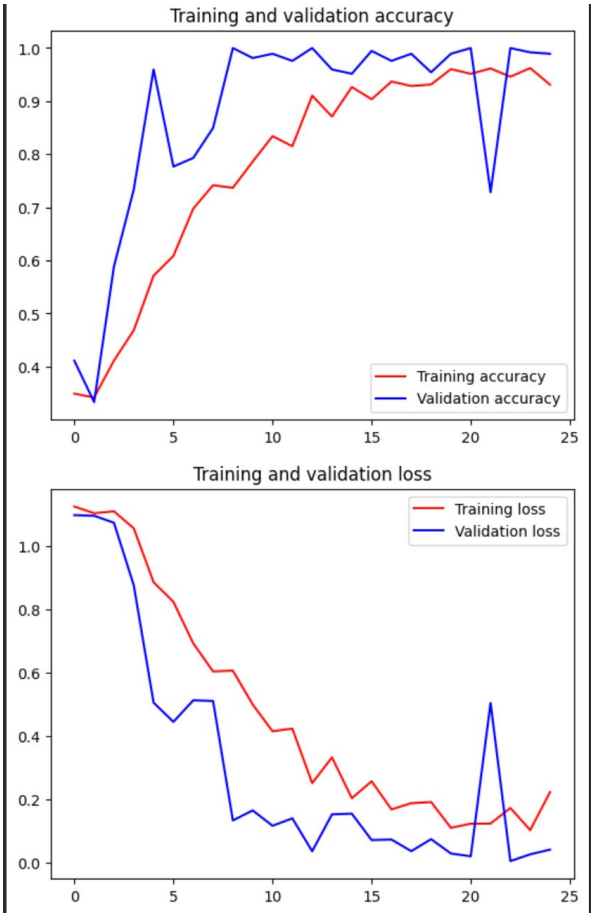
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 74, 74, 64)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dropout (Dropout)	(None, 6272)	0
dense (Dense)	(None, 512)	3,211,776
dense_1 (Dense)	(None, 3)	1,539

Total params: 3,473,475 (13.25 MB)
Trainable params: 3,473,475 (13.25 MB)
Non-trainable params: 0 (0.00 B)

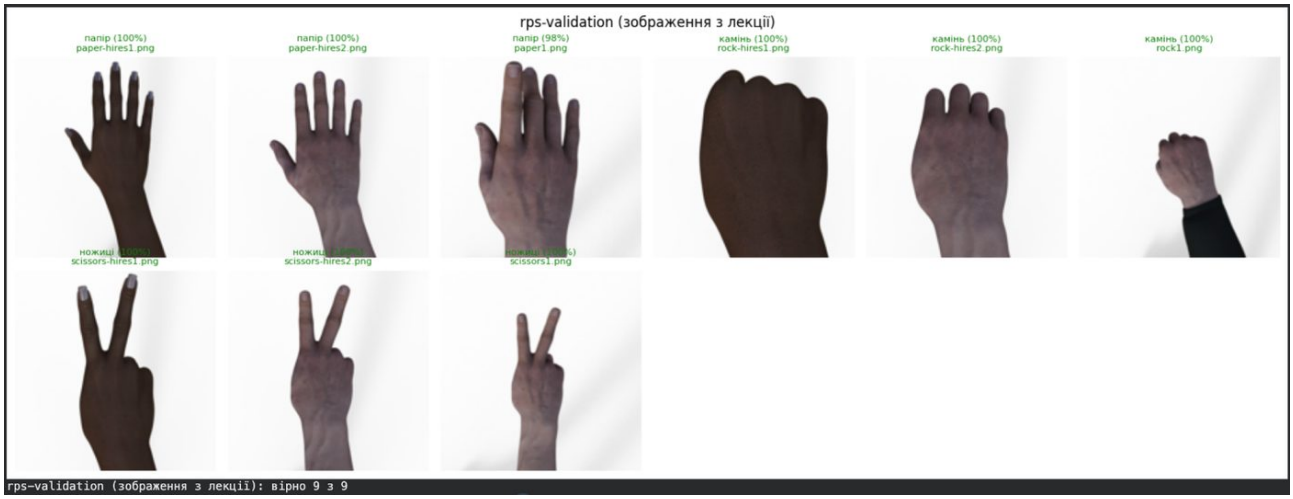
Модель: 3,47 млн параметрів, вихід Dense 3

```
Epoch 16/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9036 - loss: 0.2574 - val_accuracy: 0.9946 - val_loss: 0.0722
Epoch 17/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9369 - loss: 0.1688 - val_accuracy: 0.9758 - val_loss: 0.0738
Epoch 18/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 26s 1s/step - accuracy: 0.9286 - loss: 0.1885 - val_accuracy: 0.9892 - val_loss: 0.0372
Epoch 19/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9313 - loss: 0.1921 - val_accuracy: 0.9543 - val_loss: 0.0750
Epoch 20/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9603 - loss: 0.1107 - val_accuracy: 0.9892 - val_loss: 0.0298
Epoch 21/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9516 - loss: 0.1238 - val_accuracy: 1.0000 - val_loss: 0.0210
Epoch 22/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 29s 1s/step - accuracy: 0.9615 - loss: 0.1243 - val_accuracy: 0.7285 - val_loss: 0.5042
Epoch 23/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 28s 1s/step - accuracy: 0.9460 - loss: 0.1734 - val_accuracy: 1.0000 - val_loss: 0.0059
Epoch 24/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9623 - loss: 0.1035 - val_accuracy: 0.9919 - val_loss: 0.0272
Epoch 25/25
20/20 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.9310 - loss: 0.2234 - val_accuracy: 0.9892 - val_loss: 0.0421
```

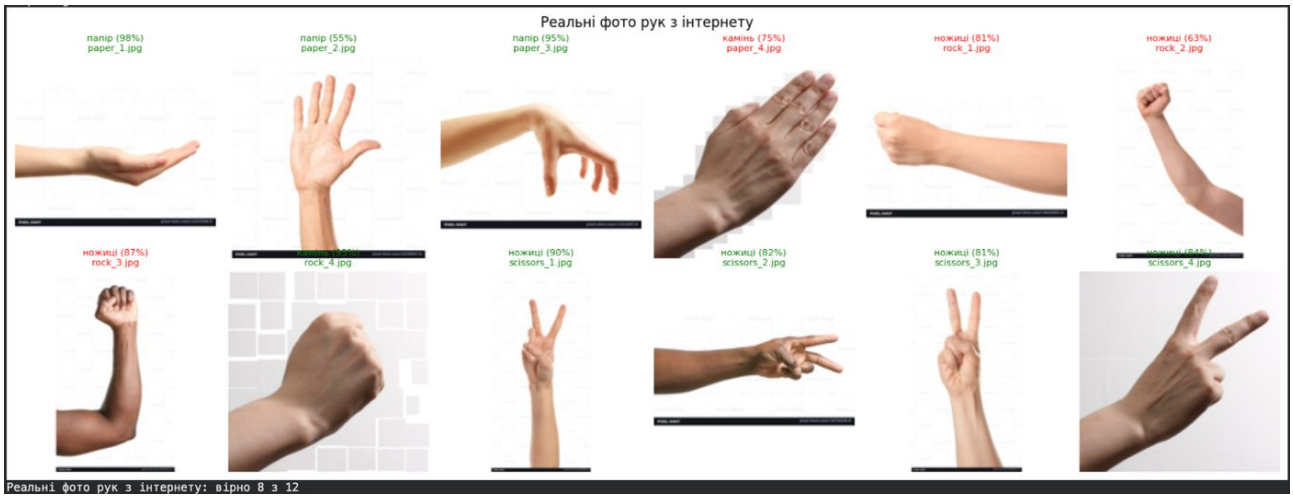
Останні 10 епох: train 93–96%, val 95–100% (на 22-й епосі разовий провал до 72,9%)



Точність і втрати (крива val вище train — через аугментацію і легкий перевірочний набір)



Пункт 15. 9 зображень rps-validation з лекції: 9 з 9



Пункт 16. 12 реальних фото рук з інтернету: 8 з 12

	файл	справжній	відповідь	папір, %	камінь, %	ножиці, %
0	paper_1.jpg	папір	папір	98.2	0.3	1.4
1	paper_2.jpg	папір	папір	55.0	3.9	41.1
2	paper_3.jpg	папір	папір	95.1	0.7	4.3
3	paper_4.jpg	папір	камінь	15.9	74.7	9.5
4	rock_1.jpg	камінь	ножиці	17.9	0.8	81.4
5	rock_2.jpg	камінь	ножиці	8.1	29.2	62.7
6	rock_3.jpg	камінь	ножиці	2.8	10.3	86.9
7	rock_4.jpg	камінь	камінь	3.4	93.3	3.3
8	scissors_1.jpg	ножиці	ножиці	6.3	3.8	89.8
9	scissors_2.jpg	ножиці	ножиці	10.0	7.5	82.5
10	scissors_3.jpg	ножиці	ножиці	12.0	7.5	80.5
11	scissors_4.jpg	ножиці	ножиці	12.6	3.3	84.1

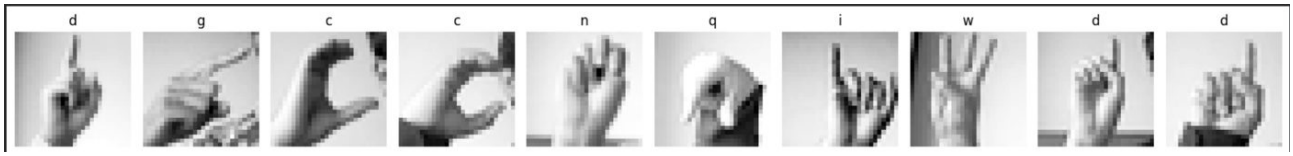
Ймовірності трьох класів для кожного фото (softmax — сума 100%)

Результат: після 25 епох train 93,1%, val **98,9%** (372 тестові фото rps-test-set), фото з лекції — 9/9 з упевненістю 98–100%. Навчальна точність нижча за перевірочну, бо навчальні фото сильно спотворені аугментацією (поворот 40°, зсув, нахил, віддзеркалення), а перевірочні — ні. **Справжні фото рук** (pixel-shot.com) — лише **8 з 12**: усі 4 «ножиці» вірні, «папір» 3/4, а «камінь» 1/4 — три кулаки мережа назвала ножицями. Причина та сама, що в коней/людей: навчали на 3D-рендерах — рука прямо, вертикально, на білому фоні. На справжніх фото рука лежить горизонтально, видно передпліччя й лікоть, інше освітлення і шкіра, тому вертикальний «стовпчик» передпліччя мережа приймає за піднятий палець. Тут, на відміну від завдання 2, немає перенесення навчання з ImageNet — мережа вчилась з нуля лише на 2520 рендерах, тож узагальнює гірше. **Додатковий експеримент з предметами** (справжні камінь, ножиці, папір — 12 власних фото) не виконано: розширення браузера не змогло завантажити файли в Colab, тому комірка вивела «папка /content/objects порожня — пропущено». Щоб виконати: завантажити фото в папку *objects* через панель «Файли» і запустити дві останні комірки.

Завдання 5. Мова жестів: Sign Language MNIST з CSV

Розв’язок — між START/END CODE HERE. `parse_data_from_input`: `csv.reader(file, delimiter=',')`, пропуск заголовка, масив float64, перший стовпець — мітки, решта 784 → reshape (N, 28, 28). Генератори: `np.expand_dims(..., -1)` (чорно-білі → 1 канал), навчальний з аугментацією (повороти 10°, зсуви/масштаб 0,1, **без віддзеркалення** — дзеркальний жест може означати іншу літеру), перевірочний — лише rescale. Модель: Conv 64 → Pool → Conv 128 → Pool → Dense 512 → Dropout 0,2 → **Dense 26 softmax**, `sparse_categorical_crossentropy` (мітки — числа, не one-hot), Adam 1e-3, 15 епох. У наборі 24 класи: J (9) і Z (25) показуються рухом.

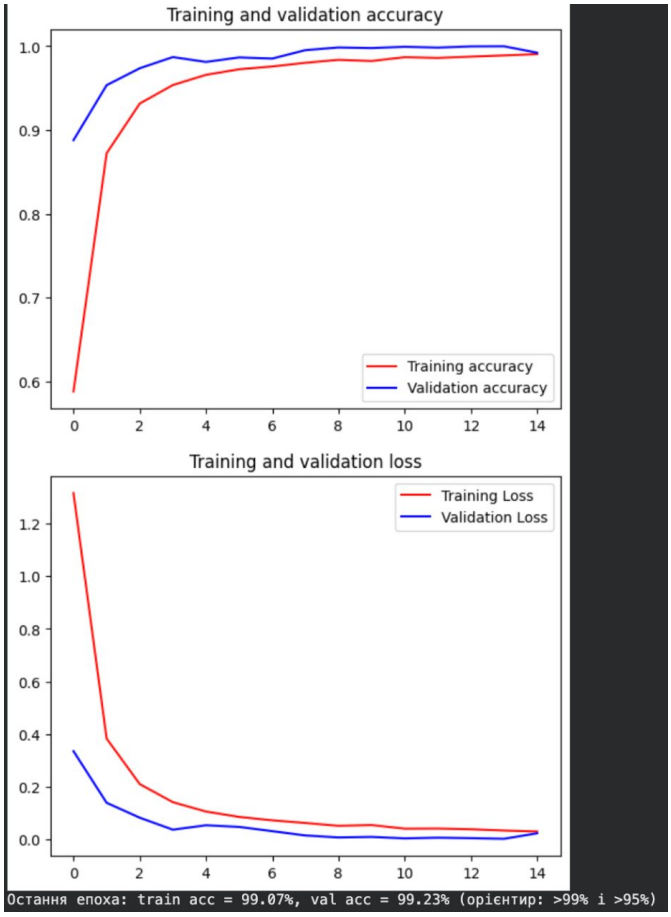
```
csv_reader = csv.reader(file, delimiter=',')
next(csv_reader)
data = np.array([row for row in csv_reader], dtype=np.float64)
labels = data[:, 0]
images = data[:, 1:].reshape(-1, 28, 28)
```



Перші 10 навчальних зображень 28×28 з літерами

```
858/858 ━━━━━━━━━━━━━━━━━━━━ 26s 22ms/step - accuracy: 0.5878 - loss: 1.3162 - val_accuracy: 0.8879 - val_loss: 0.3350
Epoch 2/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.8723 - loss: 0.3830 - val_accuracy: 0.9536 - val_loss: 0.1387
Epoch 3/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9316 - loss: 0.2099 - val_accuracy: 0.9738 - val_loss: 0.0820
Epoch 4/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9537 - loss: 0.1411 - val_accuracy: 0.9872 - val_loss: 0.0365
Epoch 5/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 16ms/step - accuracy: 0.9659 - loss: 0.1054 - val_accuracy: 0.9813 - val_loss: 0.0532
Epoch 6/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9726 - loss: 0.0851 - val_accuracy: 0.9868 - val_loss: 0.0470
Epoch 7/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9759 - loss: 0.0721 - val_accuracy: 0.9854 - val_loss: 0.0308
Epoch 8/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9803 - loss: 0.0620 - val_accuracy: 0.9954 - val_loss: 0.0146
Epoch 9/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9839 - loss: 0.0511 - val_accuracy: 0.9986 - val_loss: 0.0071
Epoch 10/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9824 - loss: 0.0540 - val_accuracy: 0.9979 - val_loss: 0.0091
Epoch 11/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9870 - loss: 0.0403 - val_accuracy: 0.9994 - val_loss: 0.0033
Epoch 12/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9862 - loss: 0.0408 - val_accuracy: 0.9985 - val_loss: 0.0061
Epoch 13/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9877 - loss: 0.0381 - val_accuracy: 0.9999 - val_loss: 0.0042
Epoch 14/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9891 - loss: 0.0331 - val_accuracy: 1.0000 - val_loss: 0.0019
Epoch 15/15
858/858 ━━━━━━━━━━━━━━━━━━━━ 13s 15ms/step - accuracy: 0.9907 - loss: 0.0296 - val_accuracy: 0.9923 - val_loss: 0.0228
```

Навчання 15 епох: ~13 с на епоху



Остання епоха: train 99,07%, val 99,23% (орієнтир методички: >99% і >95%)

Виводи функцій збігаються з Expected Output: навчальні зображення (27455, 28, 28), перевірочні (7172, 28, 28), float64; після генераторів — (27455, 28, 28, 1) і (7172, 28, 28, 1). **Результат: train 99,07%, val 99,23%** — обидві вимоги виконано (понад 99% і понад 95%), максимум на перевірці 100% на 14-й епосі. Перевірочна точність вища за навчальну майже всі епохи — аугментація і Dropout роблять навчання складнішим, ніж перевірку. Порівняно з камінь-ножиці-папір задача легша для мережі: усі фото однакового формату (28×28, рука по центру, однотонний фон) і їх у 10 разів більше (27 455).

Висновки

- **Аугментація працює як ліки від перенавчання**: на котах/собаках без неї VGG16 і InceptionV3 доходять до 99,5% на навчальних фото, а перевірочні втрати ростуть; з нею розрив train − val стає від'ємним, втрати на тесті падають (VGG16: 0,48 → 0,13; InceptionV3: 0,11 → 0,08), тест +0,4…1,2%. Ціна — навчання в 2,5 раза довше.
- **Аугментація + fine tuning — найкраща комбінація** для VGG16: 97,6% на тесті з 4000 навчальних фото, на рівні лаби 8 і на 13% краще за власну CNN з лаби 7.
- Аугментація не допомагає, коли перенавчання і так немає: ResNet50 з маленьким класифікатором (65 тис. параметрів) і коні/люди, де перевірочні фото — ті самі 3D-рендери.
- Більша роздільна здатність (300×300) **погіршила** результат: у 6 разів більше параметрів на ту саму кількість фото.

- Мультикласова класифікація відрізняється лише виходом (N нейронів softmax) і функцією втрат (categorical / sparse_categorical_crossentropy). Камінь-ножиці-папір — 98,9% на перевірці, мова жестів — 99,2%.
- Мережа, навчена на штучних 3D-рендерах, гірше розпізнає справжні фото (руки — 8/12), тоді як перенесення навчання з ImageNet (коні/люди) справляється з цим добре (14/14).